---

name: lira description: Operate the `lira` CLI — a local agent-native task system where local tickets are sub-tickets of canonical Jira parents, decomposed into atomic tasks an agent can execute. Tickets live as YAML under `~/.lira/` and can optionally bind to a peer GitHub Issue for outside visibility; Jira itself is read-only context. Use this skill whenever the user wants to break down a Jira ticket into agent work; create, claim, work, comment on, sync, or close local tickets; manage acceptance criteria and atomic tasks; resolve GitHub sync conflicts; or query the local ticket store. Trigger on phrases like "break down VAN-1234", "create a sub-ticket for", "decompose this Jira ticket", "lira ticket", "claim this", "move to in-progress", "what's next for me", "sync to GitHub", "sprint planning", or any reference to lira, agent task assignment, or local decomposition of Jira work. Also trigger whenever the workspace contains `~/.lira/` and the user is coordinating local work, even if they don't say "lira" by name.

lira — Local Jira for Agents

`lira` is a Rust CLI that gives agents a durable, inspectable, local workspace for **decomposing Jira tickets into agent-executable subtickets**. A local lira ticket is normally a child of a canonical Jira ticket: Jira holds the work item the team agreed on; lira holds the agent's local breakdown of that work into acceptance criteria and atomic tasks the agent can actually execute, comment on, and complete without touching Jira directly. Tickets live as YAML under `~/.lira/`, can optionally bind to a peer GitHub Issue for outside visibility, and never write back to Jira — Jira is read-only context in v1.

This skill covers how to drive lira from Claude Code so an agent can pull Jira context, create a local subticket, work it, and close it without violating the schema or corrupting state.

Mental model: Jira → lira → tasks

The expected hierarchy is three layers:

- **Jira ticket** (e.g. `VAN-1234`) — the canonical, team-visible work item. Read-only from lira's perspective. Holds the high-level "what" and "why."
- **lira ticket** (e.g. `ORION-48`) — the local agent's decomposition of one Jira ticket into a single agent-executable scope. Has acceptance criteria and atomic tasks. This is what the agent actually works on session-to-session.
- **lira tasks** (`T1`, `T2`, ...) — atomic to-dos inside one lira ticket. Each is small enough to finish in a single focused step.

When the user hands the agent a Jira ticket, the normal flow is:

1. `lira jira fetch VAN-1234 --json` — pull current Jira context (read-only).
2. `lira new "<local scope>" --parent-jira VAN-1234 --acceptance-criterion ... --task ...` — create the local working ticket as a Jira child.
3. Claim, move, drive tasks, comment, accumulate history.
4. Optionally `lira gh create` to mirror to a GitHub Issue so the work is visible outside the agent's local environment.
5. Close the lira ticket when its scope is done. The Jira parent stays untouched — humans update Jira separately based on the lira history and any synced GitHub Issue.

One Jira parent can have multiple lira children if the work splits across agents, sprints, or scopes. That's normal and expected — don't try to cram a multi-scope Jira ticket into one lira ticket.

A lira ticket without a Jira parent is allowed but should be the exception (e.g. agent-internal infrastructure work that has no team-level counterpart). If the user gives you work without a Jira reference, ask whether one exists before defaulting to a parentless ticket.

Preflight

Before doing real work, confirm lira is installed and the workspace exists:

```
lira --version
ls ~/.lira/ 2>/dev/null || echo "not initialized"
lira project list --json
```

If `~/.lira/` does not exist, ask the user before running `lira init` — that directory is the canonical store for **all** their tickets across every project, not scoped to the current repo. Treat it like initializing a global config.

Core operating rules

These are not stylistic preferences — lira's CLI will reject calls that violate them. Internalize them before issuing commands:

1. **Always pass `--json`.** Every command supports stable, versioned JSON output with `schema_version`, `ok`, and either `result` or `error`. Parse those fields. Human-format output is for humans and will silently change shape.
2. **Acceptance criteria are mandatory.** A ticket cannot exist without a non-empty `acceptance_criteria` list. Creation, GitHub adoption, and import all fail with `E_ACCEPTANCE_CRITERIA_REQUIRED` if missing.
3. **Tasks are mandatory.** Every ticket must have at least one entry in `tasks[]`. Same enforcement, `E_TASK_REQUIRED`.
4. **Tasks are atomic and minimal.** A task has only `id`, `title`, `status`, `tags`, `created_on`, and `last_modified`. Tasks do not have comments, assignees, descriptions, estimates, or external links. If a task needs any of that, it has outgrown task-shape — promote it to a child ticket with `lira child add`.
5. **Status lives in the directory.** A ticket's location (`~/.lira/projects/ORION/tickets/in-progress/ORION-42.yaml`) is part of the source of truth alongside the `status:` field. `lira mv` is the only safe way to change status. Never edit YAML or move files manually.
6. **No hard delete.** Cancel, archive, or release. `rm` on a ticket file bypasses history, the index, and any GitHub binding — it will leave the workspace in a state `lira doctor` flags as drift.
7. **Claim before mutating someone else's work.** `lira claim <ID> --agent <name>` fails if another agent owns the ticket. Respect that signal — don't reach for `--force` unless the user explicitly tells you to steal.

8. **The completion guard is real.** `lira mv <ID> done` fails with `E_COMPLETION_POLICY` unless acceptance criteria exist and all tasks are terminal (`done` or `cancelled`). Drive tasks to terminal first; don't bypass with `--force`.
9. **Default to a Jira parent.** A lira ticket is almost always a child of a Jira ticket (`--parent-jira VAN-1234`). Parentless tickets are a deliberate exception, not a default. If the user describes work without naming a Jira reference, ask before creating an orphan.
10. **YAML is canonical, everything else is rebuildable.** SQLite at `~/.lira/index/tickets.sqlite` and `~/.lira/gh-cache/` are caches. If they look weird, `lira reindex`, don't hand-edit.

Creating a ticket from a Jira parent

The standard flow starts by pulling the Jira parent so you have its title, description, and context to decompose against:

```
lira jira fetch VAN-1234 --json
```

Then create the local subticket with `--parent-jira` and at least one `--acceptance-criterion` and one `--task`. Both repeatable flags can appear multiple times.

```
lira new "Implement CUPED variance reduction in experiment-analyst" \
  --project ORION \
  --parent-jira VAN-1234 \
  --type task \
  --priority high \
  --assignee athena-analyst \
  --acceptance-criterion "Variance reduced by ≥30% on holdout dataset." \
  --acceptance-criterion "Report includes both baseline and adjusted variance." \
  --task "Add SQL covariate extraction." \
  --task "Implement CUPED adjustment calculation." \
  --task "Add tests for null covariates and treatment groups." \
  --json
```

The response returns the new ticket ID (e.g., `ORION-48`) and default status (`backlog`). Capture the ID for follow-up calls.

Decomposing the Jira parent

The agent's job at creation time is to translate the Jira parent's prose into two distinct artifacts:

- **Acceptance criteria** — observable, testable outcomes a reviewer could check without watching the agent work. “CSV exports include UTF-8 BOM.” “P95 latency under 100ms on 1k-ticket workspace.” These usually map directly to language already in the Jira ticket; pull from there when possible.
- **Tasks** — atomic implementation steps the agent can execute one at a time. “Add BOM to the CSV writer.” “Add benchmark script with 1k-ticket fixture.” These are the agent’s own decomposition; they typically don’t appear in Jira at all.

A criterion is what “done” looks like to an outside observer. A task is one move toward getting there. If the Jira parent is too broad to fit one lira ticket — multiple distinct outcomes, several stakeholders, work that obviously spans sprints — create **multiple sibling lira tickets** all sharing the same `--parent-jira`, each with its own focused scope, criteria, and tasks.

If the user gives only a Jira ticket key and rough direction, draft criteria and tasks yourself and show them before submitting `lira new`. Let the user edit if they want. Don’t ask the user to enumerate every task — that’s the agent’s value-add.

Picking up work

```
lira next --project ORION --agent athena-analyst --json   # what should I work on
lira claim ORION-48 --agent athena-analyst --json         # take ownership
lira mv ORION-48 in-progress --json                       # move into active stat
```

`lira next` returns the highest-priority unclaimed candidate. If none exist, the result is empty — surface that to the user; don’t invent work or claim something arbitrarily.

`lira active --agent <name> --json` lists what an agent already owns, useful at the start of a session to recover state.

Working through tasks

Each task mutation automatically appends a history entry on the parent ticket and updates `timestamps.updated`. You don’t need to manage history yourself for ordinary task moves.

```
lira task list ORION-48 --json
lira task status ORION-48 T1 in-progress --json
lira task status ORION-48 T1 done --json
lira task add ORION-48 "Document new CUPED config knobs." --tag docs --json
lira task tag add ORION-48 T2 statistics --json
```

```
lira task cancel ORION-48 T3 --json
```

Task statuses are `todo | in-progress | blocked | done | cancelled`.

When a task can't be completed as written, **don't silently retitle it**. Choose one of:

- Add a tighter sibling with `lira task add`.
- `lira task cancel` the original and add a replacement.
- Promote to a child ticket via `lira child add` if the work has outgrown atomic shape.

Cancellation is recorded in history; retitling hides the change.

Comments and history

Comments are free-form, Jira-style, human-readable notes attached to the **ticket** (not to individual tasks — tasks intentionally have no comments). Use them for context worth a human reader's attention.

History is the structured, machine-readable, append-only event stream. lira writes history automatically for mutations; agents add their own structured entries via `lira history add` when they want a durable record of an analysis step or decision.

```
lira comment ORION-48 "Variance baseline = 0.0234 on holdout slice." --json

lira history add ORION-48 \
  --action analysis_note \
  --message "CUPED assumptions validated against pre-treatment period." \
  --actor athena-analyst \
  --json
```

For long bodies, pipe via stdin:

```
lira comment ORION-48 --stdin --json <<'EOF'
Detailed multi-line note...
With supporting context.
EOF
```

If a comment should also be pushed to the bound GitHub Issue, mark it for sync:

```
lira comment sync ORION-48 local-c7 --github --json
```

GitHub comment sync is **append-only** in v1 — edits and deletes are not propagated.

Closing the ticket

```
lira mv ORION-48 in-review --json
lira mv ORION-48 done --json
```

`mv done` will fail with `E_COMPLETION_POLICY` if any task is non-terminal or if acceptance criteria are missing. Fix the underlying state — finish or cancel each task — rather than reaching for `--force`. `--force` is a human override, not an agent shortcut.

Other links and dependencies

The Jira parent is normally set at creation via `--parent-jira` (see above). Use the `link` commands for everything else: lira-to-lira parents, blocking relationships, related tickets, and child tickets.

Important distinction lira makes between two GitHub relationships:

- **GitHub *parent*** (typed parent reference, set via `--parent-github`) — an upward *tracking* relationship, usually pointing at a GitHub tracking issue that aggregates work. Local-only; not synced.
- **GitHub *peer binding*** (in the ticket's `github` block) — a 1:1 *sync* relationship with a single GitHub Issue. Bidirectional. Covered in the GitHub sync section below.

A ticket can have a Jira parent **and** a GitHub peer binding to a different GitHub Issue at the same time — they don't conflict.

```
lira link ORION-48 --jira VAN-1234                          # set/update Jira pare
lira link ORION-48 --parent-lira ORION-12                   # local lira parent
lira link ORION-48 --parent-github example-org/repo#100     # GitHub tracking pare
lira link ORION-48 --blocks ORION-50 --json
lira link ORION-48 --relates-to ORION-30 --json
lira child add ORION-48 ORION-49 --json                     # nest a local lira ch
```

For visibility into Jira-parented work:

```
lira jira sync-parents --json                               # refresh cached Jira
lira ticket list --parent-jira VAN-1234 --json              # all lira children of
```

GitHub sync

GitHub Issues are first-class peer sync targets. lira shells out to the `gh` CLI; it never

stores raw tokens. If `gh` is missing or unauthenticated, sync commands return `E_GH_NOT_INSTALLED` or `E_GH_AUTH`, and all local-only commands keep working. Don't block local progress on GitHub being available.

Binding patterns

```
# Create a fresh GitHub Issue from an existing local ticket and bind it
lira gh create ORION-48 --repo weisberg/agent_tools --json

# Bind a local ticket to an existing GitHub Issue
lira gh link ORION-48 weisberg/agent_tools#142 --json

# Adopt a single GitHub Issue into a new local ticket
lira gh adopt weisberg/agent_tools#142 \
  --project ORION \
  --acceptance-criterion "Reproduce and fix the reported regression." \
  --task "Reproduce locally from the issue body." \
  --json

# Bulk-adopt GitHub Issues
lira gh import weisberg/agent_tools \
  --project ORION \
  --state open \
  --label bug \
  --acceptance-criteria-file ./default-ac.yaml \
  --task-template "Triage imported GitHub issue." \
  --json
```

`gh adopt` and `gh import` enforce the same mandatory acceptance-criteria and tasks rules. If the GitHub body doesn't contain parseable `## Acceptance Criteria` and `## Tasks` sections, you must provide them via flags — lira refuses to create incomplete local tickets.

Push, pull, sync

```
lira gh pull ORION-48 --json                    # remote → local
lira gh push ORION-48 --json                    # local → remote
lira gh sync ORION-48 --json                    # three-way reconciliation
lira gh sync --all --project ORION --json
lira gh status ORION-48 --json
```

`sync_state` values: `synced`, `local-ahead`, `remote-ahead`, `conflict`, `unbound`, `disabled`. Always check `sync_state` before assuming either side is authoritative.

Conflict handling

`lira gh sync` **detects conflicts via three-way reconciliation against** `last_synced`**,** `remote_etag` **/** `remote_body_hash`**, and** `local_hash`**. When it returns a conflict, stop and report — do not blindly resolve.**

```
lira gh conflicts --json                                          # list conflicts
lira gh conflicts show ORION-48 --json
lira gh diff ORION-48                                             # human diff
cat ~/.lira/gh-cache/conflicts/ORION-48.diff                      # raw diff
```

Resolve only after understanding which side should win:

```
lira gh resolve ORION-48 --prefer local --json
lira gh resolve ORION-48 --prefer remote --json
```

If a human is in the loop, surface the diff and the affected fields (e.g., `["body"]`**) and ask before choosing. A common pattern: if local has tasks/AC the remote body lacks,** `--prefer local` **is usually right — but confirm.**

Search, query, board

```
lira ls --project ORION --json
lira show ORION-48 --json
lira search "token refresh" --json
lira query --status in-progress --label rust --json
lira query --task-status blocked --json              # tickets with any blocked t
lira query --task-tag sql --json
lira count --project ORION --group-by status --json
lira board --project ORION --json                    # kanban-shaped output
lira active --agent athena-analyst --json            # what I currently own
```

For large workspaces use `--limit` **and** `--cursor` **to paginate. Don't fetch everything when you only need a slice.**

Validation and recovery

```
lira doctor --json     # full health: schema, AC presence, task validity, status
lira validate --json   # focused subset: schema and transition correctness
lira reindex --json    # rebuild ~/.lira/index/tickets.sqlite from canonical YAML
```

Run `lira doctor` after anything unusual: an interrupted command, a manual edit by the user, suspected drift between status field and directory, or a `gh` operation that errored out mid-flight. The index can always be rebuilt; canonical YAML cannot.

Inspecting the filesystem directly

The directory layout is part of the product. When debugging or grepping, you can lean on it directly:

```
ls ~/.lira/projects/ORION/tickets/in-progress/                  # everything curr
cat ~/.lira/projects/ORION/tickets/in-progress/ORION-48.yaml
tail ~/.lira/logs/$(date -u +%Y-%m-%d).jsonl                    # today's mutatio
ls ~/.lira/gh-cache/conflicts/                                  # outstanding con
```

Read freely, but **write only through `lira` commands**. Direct writes bypass locks (`~/.lira/locks/`), atomic-rename semantics, and the JSONL audit log.

Common error codes

Check `error.error_code` on JSON failures. The codes you'll hit most:

Code	Meaning and response
`E_ACCEPTANCE_CRITERIA_REQUIRED`	Add `--acceptance-criterion` flags and retry.
`E_TASK_REQUIRED`	Add `--task` flags and retry.
`E_COMPLETION_POLICY`	Drive remaining tasks to `done` or `cancelled` first; don't auto-`--force`.
`E_INVALID_TRANSITION`	Run `lira project show <P> --json` to see allowed transitions for the current status.
`E_INVALID_TASK_STATUS` / `E_INVALID_TASK_SCHEMA`	Re-read the task schema rules above. Tasks have only six fields.
`E_LOCK_UNAVAILABLE`	Another process holds the lock; wait briefly and retry, or run `lira doctor` if stale.
`E_GH_NOT_INSTALLED` / `E_GH_AUTH`	Local-only ops still work. Tell the user to install `gh` or run `gh auth login`.
	Three-way conflict; read the diff and ask the user

`E_GH_CONFLICT`	before resolving.
`E_GH_RATE_LIMIT`	Back off; retry later. Don't hammer.
`E_TICKET_NOT_FOUND` / `E_PROJECT_NOT_FOUND`	Verify with `lira ls` or `lira project list`.
`E_INVALID_YAML` / `E_SCHEMA_VALIDATION`	Run `lira doctor`; usually points at a file the user edited by hand.

The full list is in PRD §18.3; the table above covers ~90% of agent-encountered failures.

Anti-patterns to avoid

- **Editing ticket YAML directly** — bypasses locks, history, and the index. Always use commands.
- **`rm`-ing a ticket file** — use `lira mv <ID> cancelled` or `lira archive <ID>`.
- **Treating tasks as separate tickets** — they live inside their parent. For separateness, use `lira child add`.
- **Skipping `--json`** — the human format will silently change shape and break parsing.
- **Forcing past the completion guard** — `--force` is a human override, not an agent default.
- **Adding comments or assignees to tasks** — that level of metadata belongs on the ticket. Promote to a child ticket if you really need it.
- **Hand-editing `~/.lira/gh-cache/`** — volatile remote state lives there for a reason; lira manages it.
- **Stealing a claim silently** — `lira claim` failing because someone else owns the ticket is a deliberate signal. Surface it; don't `--force` past it.
- **Running `lira gh sync --all` after a long hiatus without checking** — likely produces conflicts. Pull individual tickets first, or be ready to triage the conflict list.
- **Creating a parentless lira ticket when a Jira parent exists** — defeats the purpose of the tool. If the user mentions a Jira key, ticket, or even just a Jira-shaped scope, ask for the parent before creating without one.
- **Trying to write back to Jira** — Jira is read-only in v1. `lira jira fetch` and `lira jira sync-parents` only pull. Humans update Jira based on lira history and the

GitHub mirror.

- **Cramming a multi-scope Jira ticket into one lira ticket** — create siblings sharing the same `--parent-jira` instead. Each lira ticket should be one focused, completable scope.